```c
/**
 * @file axis-fifo-echo-test.c
 * @author Jason Gutel jason.gutel@gmail.com
 *
 * Simple test app that will prompt user to send an ASCII
 * string to the tx fifo in one thread and print everything
 * that shows up on the rx fifo. Useful for debugging.
 *
 * @bug No known bugs.
 **/

#include <stdlib.h>
#include <stdio.h>
#include <stdbool.h>
#include <assert.h>

#include <fcntl.h>              // Flags for open()
#include <sys/stat.h>           // Open() system call
#include <sys/types.h>          // Types for open()
#include <unistd.h>             // Close() system call
#include <string.h>             // Memory setting and copying
#include <getopt.h>             // Option parsing
#include <errno.h>              // Error codes
#include <pthread.h>
#include <signal.h>
#include <time.h>
#include <poll.h>
#include <sys/ioctl.h>

#include <sys/socket.h>
#include <netinet/in.h>
#include "../axis-fifo.h"

/*-----------------------------------------------------------------------------
 * Internal Definitions
 *-----------------------------------------------------------------------------*/
#define DEF_DEV_TX "/dev/axis_fifo_0x43c10000"
#define DEF_DEV_RX "/dev/axis_fifo_0x43c10000"
#define MAX_BUF_SIZE_BYTES 1450

#define DEBUG_PRINT(fmt, args...) printf("DEBUG %s:%d(): " fmt, \
        __func__, __LINE__, ##args)

struct thread_data {
    int rc;
};

pthread_t write_to_fifo_thread;
pthread_t read_from_fifo_thread;

static volatile bool running = true;
static char _opt_dev_tx[255];
static char _opt_dev_rx[255];
static int writeFifoFd;
static int readFifoFd;

static void signal_handler(int signal);
static void *read_from_fifo_thread_fn(void *data);
static int process_options(int argc, char * argv[]);
static void print_opts();
static void display_help(char * progName);
static void *write_to_fifo_thread_fn(void *data);
static void quit(void);

/*-----------------------------------------------------------------------------
 * Main
 *-----------------------------------------------------------------------------*/
int main(int argc, char **argv)
{
    process_options(argc, argv);

    int rc;

    // Listen to ctrl+c and assert
    signal(SIGINT, signal_handler);
```

```
    signal(SIGTERM, signal_handler);
    signal(SIGQUIT, signal_handler);

    /*************/
    /* open FIFO */
    /*************/
    readFifoFd = open(_opt_dev_rx, O_RDONLY | O_NONBLOCK);
    writeFifoFd = open(_opt_dev_tx, O_WRONLY);
    if (readFifoFd < 0) {
        printf("Open read failed with error: %s\n", strerror(errno));
        return -1;
    }
    if (writeFifoFd < 0) {
        printf("Open write failed with error: %s\n", strerror(errno));
        return -1;
    }

    /*****************************/
    /* initialize the fifo core  */
    /*****************************/
    rc = ioctl(readFifoFd, AXIS_FIFO_RESET_IP);
    if (rc) {
        perror("ioctl");
        return -1;
    }
    rc = ioctl(writeFifoFd, AXIS_FIFO_RESET_IP);
    if (rc) {
        perror("ioctl");
        return -1;
    }

    /*****************/
    /* start threads */
    /*****************/

    /* start thread listening for ethernet packets */
    rc = pthread_create(&write_to_fifo_thread, NULL, write_to_fifo_thread_fn,
            (void *)NULL);

    /* start thread listening for fifo receive packets */
    rc = pthread_create(&read_from_fifo_thread, NULL, read_from_fifo_thread_fn,
            (void *)NULL);

    /* perform noops */
    while (running) {
       sleep(1);
    }

    printf("SHUTTING DOWN\n");
    pthread_join(write_to_fifo_thread, NULL);
    pthread_join(read_from_fifo_thread, NULL);
    close(writeFifoFd);
    close(readFifoFd);
    return rc;
}

static void quit(void)
{
    running = false;
}

static void *write_to_fifo_thread_fn(void *data)
{
    int rc;
    int packets_tx;
    ssize_t bytesFifo;
    char buf[MAX_BUF_SIZE_BYTES];
    uint32_t vacancy;

    /* shup up compiler */
    (void)data;

    packets_tx = 0;

    while (running) {
```

```
        do {
            rc = ioctl(writeFifoFd, AXIS_FIFO_GET_TX_VACANCY, &vacancy);
            if (rc) {
                perror("ioctl");
                return (void *)0;
            }
            if (vacancy < (uint32_t)MAX_BUF_SIZE_BYTES)
                usleep(100);
        } while (vacancy < (uint32_t)MAX_BUF_SIZE_BYTES);

        printf("Send a message to %s : ",_opt_dev_tx);
        scanf("%s",&buf[0]);
        printf("Sending %s\n\r",buf);

        bytesFifo = write(writeFifoFd, buf, strlen(buf));
        if (bytesFifo > 0) {
            DEBUG_PRINT("bytes to fifo %d\n",bytesFifo);
            packets_tx++;
        } else {
            perror("write");
            quit();
        }
    }

    return (void *)0;
}

static void *read_from_fifo_thread_fn(void *data)
{
    ssize_t bytesFifo;
    int packets_rx;
    uint8_t buf[MAX_BUF_SIZE_BYTES];

    /* shup up compiler */
    (void)data;

    packets_rx = 0;

    while (running) {
        bytesFifo = read(readFifoFd, buf, MAX_BUF_SIZE_BYTES);
        if (bytesFifo > 0) {
            DEBUG_PRINT("bytes from fifo %d\n",bytesFifo);
            DEBUG_PRINT("Read : %s\n\r",buf);
            packets_rx++;
        }
    }

    return (void *)0;
}

static void signal_handler(int signal)
{
    switch (signal) {
        case SIGINT:
        case SIGTERM:
        case SIGQUIT:
            running = false;
            break;

        default:
            break;
    }
}

static void display_help(char * progName)
{
    printf("Usage : %s [OPTIONS]\n"
           "\n"
           "  -h, --help     Print this menu\n"
           "  -t, --devTx    Device to use ... /dev/axis_fifo_0x43c10000\n"
           "  -r, --devRx    Device to use ... /dev/axis_fifo_0x43c10000\n"
           ,
           progName
          );
}
```

```
static void print_opts()
{
    printf("Options : \n"
            "DevTX          : %s\n"
            "DevRx          : %s\n"
           ,
           _opt_dev_tx,
           _opt_dev_rx
          );
}

static int process_options(int argc, char * argv[])
{
        strcpy(_opt_dev_tx,DEF_DEV_TX);
        strcpy(_opt_dev_rx,DEF_DEV_RX);

        for (;;) {
            int option_index = 0;
            static const char *short_options = "hr:t:";
            static const struct option long_options[] = {
                    {"help", no_argument, 0, 'h'},
                    {"devRx", required_argument, 0, 'r'},
                    {"devTx", required_argument, 0, 't'},
                    {0,0,0,0},
                    };

            int c = getopt_long(argc, argv, short_options,
            long_options, &option_index);

            if (c == EOF) {
            break;
            }

            switch (c) {
                case 't':
                    strcpy(_opt_dev_tx, optarg);
                    break;

                case 'r':
                    strcpy(_opt_dev_rx, optarg);
                    break;

                default:
                case 'h':
                    display_help(argv[0]);
                    exit(0);
                    break;
                    }
            }

        print_opts();
        return 0;
}
```